

# The Database Book

How five tables, two functions, and one trigger decide which student sees which flashcard — every column, constraint, and policy explained.

- 01 students
- 02 products
- 03 purchases
- 04 chapters
- 05 flashcards
- 06 access

## WHERE THINGS STAND TODAY

- LIVE

All five tables created, RLS enabled on every one of them.
- LIVE

`handle_new_student` trigger — copies signups into `students` .
- LIVE

`has_class_access` function — ran successfully in the SQL Editor.
- TO DO

The new `flashcards` policy. Until it runs, every logged-in user can read every card.
- TO DO

Prefixed chapter slugs, `on update cascade` on the flashcards FK, and the two React guards.

00

Why the database must decide

01

students — and the signup trigger

02

products — the catalog

03

purchases — the ledger

04

chapters — the content spine

05

flashcards — the paid goods

06

`has_class_access` — the bridge

07

What happens during one request

08

The React guards, and their limits

09

Adding the next content table

A

Glossary of every keyword used

B

Open items and known gaps

CHAPTER 00

## Why the database must decide

Your React app talks to Supabase directly from the browser. To do that it carries the **anon key**, and that key is inside your JavaScript bundle. Anyone can open DevTools and read it. That single fact determines your entire security design.

It means a stranger can skip your app entirely — no React, no router, no components — and send requests straight to your Supabase URL using that key plus a free account. Every guard written in JavaScript is invisible to them.

#### THE RULE EVERYTHING FOLLOWS

Anything the browser is capable of requesting, assume a stranger *will* request. The only thing that can refuse them is the database itself.

## Two kinds of lock, and only one is real

Your app has two layers that both look like security. They are not the same thing.

| LAYER                                                   | LIVES IN                  | WHAT IT ACTUALLY DOES                                                                                                                                                  |
|---------------------------------------------------------|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Route guards</b><br>ProtectedRoute,<br>PurchaseRoute | Your JavaScript<br>bundle | Controls what the app <i>displays</i> . Good UX — sends unpaid users away instead of showing a broken page. Zero security value: it is bypassed by not using your app. |
| <b>Row Level Security</b><br>policies on each table     | PostgreSQL                | Controls what the database <i>hands over</i> . Cannot be bypassed, because it runs after the request arrives, inside the server.                                       |

Both are worth having. Just never confuse them: **guards decide what is shown, RLS decides what exists.**

## What Row Level Security actually is

RLS is a PostgreSQL feature roughly a decade old — not a Supabase invention. It works in two steps.

### Step one: turn it on.

```
alter table flashcards enable row level security;
```

This flips the table to **deny by default**. With RLS on and no policies written, nobody can read anything — you get zero rows, not an error. That silence is the single most confusing thing about RLS, and it is also what makes it safe: forget to write a policy and content disappears, rather than leaking.

### Step two: write a policy that hands some access back.

```
create policy "a name for it"  
on <table>  
for select          -- which action: select / insert / update / delete  
to authenticated    -- which role: anon, authenticated, ...  
using (<a true/false test>);
```

The `using` expression is the heart of it. Postgres runs that test **once for every row** it considers returning. True means you see the row. False means the row simply does not exist as far as you are concerned — no error message, just fewer rows.

#### CONSEQUENCE WORTH MEMORISING

A policy that is too strict looks exactly like "there is no data." A policy that is too loose looks exactly like "everything works." That asymmetry is why you must test with a second account that owns nothing — a passing test on your own account proves nothing.

#### Actions are separate

A policy covers one action. `for select` grants reading only. Since RLS denies by default, a table with only a select policy **cannot be written to by the browser at all** — no inserts, no updates, no deletes. That is the situation for all five of your tables, and it is deliberate: your content and your purchase records are written by you in the Supabase Table Editor, never by a client.

## students — and the signup trigger

One row per signed-up student. This table is the join between Supabase's authentication system and everything you built.

| COLUMN     | TYPE       | WHAT IT MEANS                                                                         |
|------------|------------|---------------------------------------------------------------------------------------|
| id         | uuid       | Primary key <i>and</i> a foreign key to <code>auth.users(id)</code> . Cascade delete. |
| full_name  | text       | Nullable — null when signup metadata carried no name.                                 |
| email      | text       | Not null.                                                                             |
| phone      | text       | Optional, for a future profile page.                                                  |
| created_at | timestampz | Defaults to <code>now()</code> .                                                      |

### The most important line in the whole schema

```
id uuid primary key references auth.users (id) on delete cascade
```

Three separate decisions are packed into that one line.

- **primary key** — one row per student, and the id is how everything else points here.
- **references auth.users (id)** — the id must already exist as a real authenticated account. You cannot invent a student. Because the ids are *the same value*, not a separate lookup, `auth.uid()` can be compared directly against `students.id` and against `purchases.student_id`. Every access check you write depends on this.
- **on delete cascade** — delete the auth user and this row goes too, which in turn cascades to their purchases. No orphaned data.

#### WHY THIS DESIGN IS RIGHT

The alternative — a separate `user_id` column beside a surrogate `student_id` — creates two ids for one person and an opportunity for them to disagree. Sharing the primary key makes disagreement impossible.

### The policy

```
create policy "students can view own row"
on students for select
using (auth.uid() = id);
```

`auth.uid()` is a function Supabase provides inside the database. It reads the JWT that came with the request and returns that user's id. It is not something the client can set — it comes from a signed token — which is why it is trustworthy.

So the test reads: *you may see this row only if its id is you*. Every other student's row is invisible.

#### SMALL INCONSISTENCY

This policy has no `to authenticated` clause, so it applies to *all* roles including `anon`. It is still safe — for an anonymous request `auth.uid()` is null, and `null = id` evaluates to null, which is not true, so no rows come back. But your other four tables do name their role explicitly. Adding `to authenticated` here would make the intent match the rest.

## The trigger: how a row gets created

There is no `insert` policy on `students`. The browser therefore cannot create a student row — ever. So how does one appear at signup? A trigger does it.

```
create function public.handle_new_student()
returns trigger as $$
begin
  insert into public.students (id, full_name, email)
  values (new.id, new.raw_user_meta_data->'full_name', new.email);
  return new;
end;
$$ language plpgsql security definer;

create trigger on_auth_user_created
after insert on auth.users
for each row execute function public.handle_new_student();
```

#### READING IT PIECE BY PIECE

- `returns trigger` — this function is not called by you. Postgres calls it automatically when the event fires.
- `after insert on auth.users` — the event is "a new account was created in Supabase Auth." *After* means the auth row is already safely written before your code runs.
- `for each row` — run once per inserted row, not once per statement. Inside the function, `new` is that freshly created auth user.
- `new.raw_user_meta_data->'full_name'` — `raw_user_meta_data` is a JSON column holding whatever extra fields were passed at signup. The `->` operator extracts a key *as text*. Its sibling `->` would return JSON instead. If the key is absent you get null — which is why `full_name` is nullable.
- `language plpgsql` — a procedural language with `begin ... end` blocks and variables, unlike plain `sql` functions which are a single expression. Triggers need this.
- `return new` — required boilerplate for row-level triggers; it hands the row back to Postgres.
- `security definer` — the crucial one. The function runs with its *owner's* permissions instead of the caller's. Without it, the insert would be blocked, because `students` has RLS on and no insert policy. This is how a row gets in through a door that is locked to everyone.

#### THE PATTERN TO NOTICE

No insert policy plus a `security definer` trigger means the *only* way a student row can exist is via a genuine signup. Not through your app, not through a crafted request. This is the same shape you use for purchases: lock the door completely, then open exactly one controlled path.

#### HARDENING THIS ONE IS WORTH DOING

This function is `security definer` but does **not** set `search_path`. That combination is a known PostgreSQL footgun — an unqualified table name inside a definer function can be resolved against a schema an attacker controls. You are partly protected here because the names are written fully qualified (`public.students`), but adding `set search_path = public` closes the gap properly. Supabase's own database linter flags exactly this pattern. Your newer `has_class_access` already does it correctly.

## products — the catalog

What is for sale, and what buying it opens. A small table that carries a lot of design weight, because the column `unlocks_class` is the entire link between money and content.

| COLUMN                     | TYPE          | WHAT IT MEANS                                                                           |
|----------------------------|---------------|-----------------------------------------------------------------------------------------|
| <code>product_code</code>  | text          | Primary key. A readable slug like <code>class9-premium</code> , constrained by a regex. |
| <code>name</code>          | text          | Display name. Not null.                                                                 |
| <code>description</code>   | text          | Optional marketing copy.                                                                |
| <code>price</code>         | numeric(10,2) | Not null, must be $\geq 0$ .                                                            |
| <code>unlocks_class</code> | text          | Which class dashboard this opens. Same allowed values as <code>chapters.class</code> .  |
| <code>currency</code>      | text          | Defaults to <code>'INR'</code> ; only INR permitted for now.                            |
| <code>access_until</code>  | timestampz    | Not null — when access granted by this product ends.                                    |
| <code>is_active</code>     | boolean       | Default true. False hides it from browsing.                                             |
| <code>created_at</code>    | timestampz    | Defaults to <code>now()</code> .                                                        |

### A natural key instead of a number

`product_code` is the primary key — there is no numeric id. That is deliberate: the code is readable wherever it appears, so a purchase row says `class9-premium` rather than `7`, and you can understand your own data without joining tables.

```
product_code text primary key
check (product_code ~ '^[a-z0-9]+(-[a-z0-9]+)*$')
```

`~` is Postgres's regex-match operator. Reading the pattern: `^` start, one or more lowercase letters or digits, then zero or more groups of "a hyphen followed by more letters or digits", then `$` end. So `class9-premium` passes; `Class9-Premium`, `class9-`, and `-class9` are all rejected. The database enforces your naming convention so a typo cannot become a permanent key.

### Money columns

`numeric(10,2)` means up to 10 total digits with exactly 2 after the decimal point — exact decimal arithmetic. **Never use `float` or `real` for money:** binary floating point cannot represent 0.1 exactly, so totals drift by fractions of a rupee and reconciliation becomes impossible.

### The policy

```
create policy "read active products"
on products for select
to anon, authenticated
using (is_active = true);
```

Note `to anon, authenticated` — this is the one table readable while logged out, because a pricing page must work for visitors who have not signed up. That is correct: nothing here is secret. The `is_active` filter hides retired products from browsing.

#### REMEMBER THIS FOR CHAPTER 06

That `is_active = true` filter is precisely why `has_class_access` must be `security definer`. If the function ran under the caller's own permissions, this policy would hide any retired product — and every student who already bought it would instantly lose access to content they paid for. Retiring a product must stop *new sales*, not revoke *past purchases*.

## The seeded row

```
insert into products (product_code, name, description, price, unlocks_class, access_until)
values ('class9-premium', 'Class 9 Premium 2027',
       'Complete premium access for Class 9 students.',
       499.00, 'class9', '2027-03-31 23:59:59+05:30');
```

The timestamp ends in `+05:30` — an explicit IST offset. Because the column is `timestampz`, Postgres stores it as an absolute moment in UTC and converts on the way out. Always write the offset; a bare timestamp is ambiguous and will drift by hours depending on server settings.

#### THIS IS WHY ADDING A CLASS IS CHEAP

Selling Class 10 means inserting one row here with `unlocks_class = 'class10'`. No schema change, no policy change, no code change. That property is worth protecting as the project grows.



## purchases — the ledger

One row per buying *event*. One student can have many. This is the table your access decisions are ultimately made from.

| COLUMN            | TYPE          | WHAT IT MEANS                                                              |
|-------------------|---------------|----------------------------------------------------------------------------|
| id                | bigint        | Primary key, auto-numbered via <code>generated always as identity</code> . |
| student_id        | uuid          | FK → <code>students.id</code> , cascade delete.                            |
| product_code      | text          | FK → <code>products.product_code</code> , <b>on update cascade</b> .       |
| amount_paid       | numeric(10,2) | The price actually charged, frozen at purchase time. $\geq 0$ .            |
| currency          | text          | Default <code>'INR'</code> .                                               |
| payment_status    | text          | pending / completed / failed / refunded. Defaults to pending.              |
| gateway_order_id  | text          | Payment gateway's order id. Unique where present.                          |
| payment_reference | text          | Gateway reference. Unique.                                                 |
| created_at        | timestampz    | When the row appeared. Default <code>now()</code> .                        |
| purchased_at      | timestampz    | When payment completed. No default — null until it does.                   |
| expires_at        | timestampz    | Optional per-purchase expiry. Null means no expiry.                        |

### Two different cascades — and they are not the same

```
student_id  uuid references students(id)          on delete cascade
product_code text references products(product_code) on update cascade
```

- **on delete cascade** — delete the student, their purchases vanish with them.
- **on update cascade** — if a *product code is renamed*, every purchase pointing at it is renamed automatically. Without this, renaming would be rejected while any purchase referenced the old code.

Remember this pair. In chapter 05 you will meet a foreign key that has the first but is missing the second, and that omission is currently blocking a change you need to make.

### Why amount\_paid exists when products.price already does

Prices change. If you raise Class 9 Premium from ₹499 to ₹599, every historical purchase must still show what was actually charged. Copying the figure at purchase time — rather than joining to the live price — keeps your records truthful. The same reasoning applies to expiry: copy `products.access_until` into `purchases.expires_at` when the sale completes, so changing the catalog never alters what someone already bought.

## Three timestamps, three distinct meanings

| COLUMN       | SET WHEN                    | ANSWERS                         |
|--------------|-----------------------------|---------------------------------|
| created_at   | Row is inserted (automatic) | When did this attempt begin?    |
| purchased_at | Payment succeeds (manual)   | When did money actually arrive? |
| expires_at   | Payment succeeds (manual)   | When does this access end?      |

A pending or failed attempt has a `created_at` but no `purchased_at`. That distinction is what lets you tell "someone tried and the payment failed" from "someone bought."

## Status as a check constraint

```
payment_status text not null default 'pending'  
check (payment_status in ('pending', 'completed', 'failed', 'refunded'))
```

A typo like `'complete'` is rejected at write time. This matters enormously for security: `has_class_access` tests `payment_status = 'completed'`, and if a misspelled status could be stored, a paying customer would silently lose access. The constraint guarantees the values the function checks against are the only values that exist.

Defaulting to `'pending'` is the safe direction — a half-written purchase grants nothing.

## The partial unique index — the cleverest line in your schema

```
create unique index idx_student_product_completed  
on purchases(student_id, product_code)  
where payment_status = 'completed';
```

An ordinary unique index on `(student_id, product_code)` would say "a student may have only one row per product" — which breaks the moment someone's card is declined and they retry, because the failed row would block the successful one.

The `where` clause makes the index **partial**: uniqueness is enforced only among rows where the status is completed. So a student may accumulate any number of failed and pending attempts, but can hold exactly one completed purchase of a given product. Retrying works; double-charging cannot happen.

The same technique appears on `gateway_order_id` — unique *where not null*, so many rows may have no order id yet while real ids stay unique.

## Indexes

Three plain indexes exist on `student_id`, `product_code`, and `payment_status`. An index is a lookup structure that turns "scan every row" into "jump straight there." `idx_purchases_student` is the one that matters most for you — it is what makes `has_class_access` fast, since every call filters on `student_id = auth.uid()`.

## The policy — and the policy that is deliberately absent

```
create policy "read own purchases"  
on purchases for select  
to authenticated  
using (student_id = auth.uid());
```

You can read your purchases and nobody else's. This is also why `App.jsx` can query purchases without filtering by user — it is impossible to see another person's rows, so the filter would be redundant.

### THE ABSENCE IS THE FEATURE

There is **no insert policy** on `purchases`. A browser cannot create a purchase row, so nobody can grant themselves access by crafting a request. Today you add rows by hand in the Table Editor after confirming payment. When a payment gateway is wired up, its webhook will run server-side with the service role key — which bypasses RLS entirely — and this table's policies will not need to change at all.

## chapters — the content spine

One row per chapter of study material. Small table, but it is the hinge the whole access system turns on: it is the only place that records which class a piece of content belongs to.

| COLUMN  | TYPE | WHAT IT MEANS                                                                                 |
|---------|------|-----------------------------------------------------------------------------------------------|
| slug    | text | Primary key. Readable key used in URLs and referenced by flashcards.                          |
| class   | text | Not null. One of class6–class12 or neet. <b>This is the value access is decided on.</b>       |
| subject | text | Not null. physics / chemistry / biology / english / socialscience / maths / zoology / botany. |
| title   | text | Display name, e.g. "Particulate Nature of Matter".                                            |

### The check constraints keep three systems in agreement

The allowed values for `class` are exactly the allowed values for `products.unlocks_class`, and also exactly the `:classId` segments in your URLs. Three places, one vocabulary. If they ever drifted apart — say a chapter stored `'9'` while a product stored `'class9'` — the comparison inside `has_class_access` would quietly return false and paying students would see nothing. The check constraints are what prevent that drift.

### Why this table stays readable by everyone

```
create policy "authenticated can read chapters"
on chapters for select
to authenticated
using (true);
```

`using (true)` means every logged-in user sees every chapter. On `flashcards` that same line is a serious leak — here it is fine, and the reason is worth understanding rather than memorising.

Chapter titles and slugs are **already public**. They ship inside your JavaScript bundle in `src/data/`, because that is what draws the chapter list. Locking them in the database would protect nothing that is not already downloadable. What is *not* in the bundle — the questions and answers — is what needs the lock.

#### A CONSEQUENCE TO KEEP IN MIND

Because chapters stay open, a student without the purchase can still read a chapter's *title*. This has a practical effect covered in chapter 07: a blocked request returns the chapter row with an empty card list, rather than failing outright.

### The slug problem

Your schema comment says slugs look like `class9-physics-motion`. Your actual data uses bare slugs — `src/data/class8/chapters/chemistry8.js` holds `particulate-nature-of-matter`.

`slug` is the **primary key**, which means globally unique across every class and subject. Bare slugs therefore collide the moment two classes share a chapter name — and NCERT repeats names constantly. Class 9 physics "Motion" and Class 11 physics "Motion" both want `motion`; only one can exist. NEET overlaps classes 11 and 12 almost entirely.

#### **FIX THIS BEFORE SEEDING MORE CONTENT**

The recommended fix keeps bare slugs in your data files and URLs — so links stay short and no data file needs editing — and stores *prefixed* slugs in the database, composing the full key at query time from `classId`, `subject`, and `chapterSlug`, all three of which are already in the route. Security is unaffected either way, because RLS reads `chapters.class` off the row it found rather than trusting anything composed by the client.

## flashcards — the paid goods

The question-and-answer cards. This is the only table in your database holding something a person actually pays for, which makes it the only table where a policy mistake costs money.

| COLUMN       | TYPE   | WHAT IT MEANS                                               |
|--------------|--------|-------------------------------------------------------------|
| id           | bigint | Primary key, auto-numbered.                                 |
| chapter_slug | text   | FK → <code>chapters.slug</code> , cascade delete. Not null. |
| question     | text   | Front of the card. Not null.                                |
| answer       | text   | Back of the card. Not null.                                 |
| position     | int    | Ordering within the chapter. Default 0, must be $\geq 0$ .  |

### not null on chapter\_slug is a security property

Every card must belong to a chapter. That is not merely tidy — the access policy works by looking up the card's chapter to find its class. A card with a null chapter would have *no class*, and therefore no way to be checked. `not null` plus the foreign key guarantees the lookup always succeeds, so there is no orphan row that could slip past the policy.

### The unique (chapter\_slug, position) rule

No two cards in the same chapter may occupy the same slot, though position 3 may exist in many different chapters. This makes ordering deterministic — which matters because `FlashcardListPage` requests `.order("position", { foreignTable: "flashcards" })`, and without a stable order the deck would shuffle unpredictably between loads.

### The missing cascade

```
chapter_slug text not null
  references chapters(slug)
  on delete cascade,      -- present
                        -- on update cascade is NOT here
```

Deleting a chapter removes its cards, which is right. But **renaming** a chapter's slug will be rejected while any flashcard points at it. Since chapter 04 concluded you should rename slugs to a prefixed form, this omission blocks that work — add `on update cascade` to this constraint first. Your purchases table already uses it on `product_code`; this is the same idea applied to content.

### The policy: before and after

The original policy shipped as:

```
create policy "authenticated can read flashcards"
  on flashcards for select
  to authenticated
  using (true);      -- every logged-in user, every card
```

Signing up is free, so this makes paid content free to anyone who registers. The replacement:

```
create policy "read flashcards for purchased class"
on flashcards for select
to authenticated
using (
  exists (
    select 1 from chapters
    where chapters.slug = flashcards.chapter_slug
    and has_class_access(chapters.class)
  )
);
```

In English: *for this card, find its chapter; ask whether the reader has access to that chapter's class.*

- `exists ( ... )` returns true or false — exactly what `using` needs. You are asking whether a matching row exists, not fetching it.
- `select 1` is the convention inside `exists`. The value is never used, so there is no point selecting real columns.
- `chapters.slug = flashcards.chapter_slug` correlates the subquery with the row being tested. `flashcards` here refers to the current row under evaluation.
- `has_class_access(chapters.class)` is the call. **This is where the class value comes from** — read off the chapter row, never from the request.

#### WHY IT IS FAST

`chapters.slug` is the primary key, so the lookup is an index seek rather than a scan, and `idx_flashcards_chapter` supports the other side. Marking the function `stable` lets Postgres reuse one answer for all cards in the same class instead of re-running it per card.

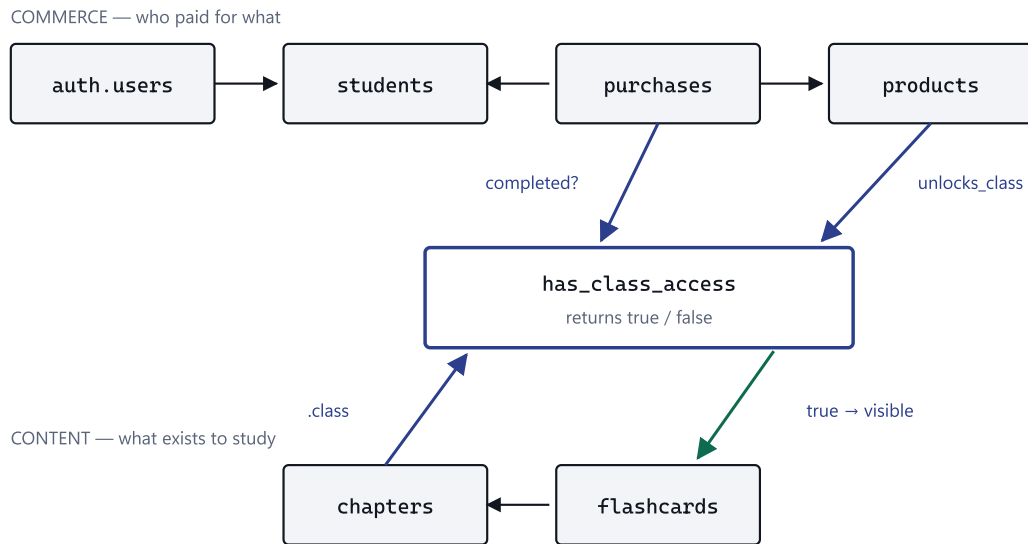
#### FOLLOW YOUR OWN CONVENTION WHEN YOU APPLY THIS

Your `database/migrations/README.md` says: never edit files in `schema/` retroactively — those describe the schema as first created — and record every later change as a numbered migration. This policy swap is exactly such a change, so it belongs in a new file like `database/migrations/001_flashcards_purchase_policy.sql` containing the `drop policy` and `create policy` statements, rather than being edited into `05_flashcards.sql`.

## has\_class\_access — the bridge

Your database has two groups of tables that never touch. Commerce: students, purchases, products. Content: chapters, flashcards. No foreign key joins them, and none should — content exists whether or not anyone buys it.

This function is the only bridge between them.



Two independent chains. The function is the only thing that reads from both, which is why every access rule you will ever change lives inside it.

### The function

```
create or replace function has_class_access(target_class text)
returns boolean
language sql
stable
security definer
set search_path = public
as $$
    select exists(
        select 1
        from purchases
        join products on products.product_code = purchases.product_code
        where purchases.student_id = auth.uid()
            and purchases.payment_status = 'completed'
            and products.unlocks_class = target_class
            and (purchases.expires_at is null or purchases.expires_at > now())
    );
$$;
```



## Every keyword, and why it is there

| KEYWORD                               | MEANING, AND WHY THIS ONE                                                                                                                                                             |
|---------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>create or replace</code>        | Creates it, or overwrites an existing one of the same name. Lets you re-run the file while editing without dropping first.                                                            |
| <code>(target_class text)</code>      | A parameter — an empty slot filled by whoever calls the function. Nothing is stored in it.                                                                                            |
| <code>returns boolean</code>          | Gives back true or false, exactly the shape a <code>using</code> clause needs.                                                                                                        |
| <code>language sql</code>             | The body is one plain SQL statement — no variables or control flow needed. Simpler and faster than <code>plpgsql</code> , which the signup trigger needs because triggers require it. |
| <code>stable</code>                   | A promise that the answer will not change within a single query for the same input. Lets Postgres call it once per distinct class instead of once per flashcard row.                  |
| <code>security definer</code>         | Runs with the owner's permissions rather than the caller's, so it can see rows the caller's own policies would hide.                                                                  |
| <code>set search_path = public</code> | Pins which schema unqualified table names resolve to. Mandatory hygiene alongside <code>security definer</code> .                                                                     |
| <code>\$\$ ... \$\$</code>            | "Dollar quoting" — marks the start and end of the body so you can write quotes inside without escaping them.                                                                          |

### ON VOLATILITY: STABLE, AND ITS SIBLINGS

Postgres wants to know how predictable a function is, so it can decide how often to run it.

- **volatile** (the default) — may return something different every call, like `random()`. Postgres must call it for every row.
- **stable** — consistent within one statement, but may differ between statements. Anything reading tables belongs here, because the data can change between queries. This is your case.
- **immutable** — same input always gives the same output, forever, like `lower('ABC')`. Marking a table-reading function immutable would be a bug: Postgres could cache a stale answer indefinitely, meaning a refunded student might keep access.

### ON SECURITY DEFINER, HONESTLY

The function reads two tables that have their own policies. Under the caller's permissions, `purchases` would be filtered to their own rows — which is what you want anyway — but `products` would be filtered to `is_active = true`. The day you retire a product, everyone who bought it would lose access to content they paid for. `security definer` is what prevents that.

It is a real privilege escalation, so it comes with a rule:

#### NEVER BREAK THIS

The function must never take a user id as a parameter. Identity comes from `auth.uid()` inside the body, where the caller cannot influence it. A signature like `has_class_access(user_id uuid, class text)` would let anyone ask about anyone — and because the function is `security definer`, it would answer.

## ON SEARCH\_PATH

When SQL says `purchases` without a schema, Postgres searches a list of schemas in order. If someone can create a schema earlier in that list containing their own table called `purchases`, a definer function would read *their* table and happily return true for everyone. Pinning `search_path = public` removes the ambiguity. It costs one line and closes a whole class of attack.

## The four conditions in the body

| CONDITION                                     | WHAT BREAKS WITHOUT IT                                                                            |
|-----------------------------------------------|---------------------------------------------------------------------------------------------------|
| <code>student_id = auth.uid()</code>          | Every student would inherit everyone else's purchases. One person buys, everyone unlocks.         |
| <code>payment_status = 'completed'</code>     | A merely <i>attempted</i> payment would unlock content. Refunds would never revoke access.        |
| <code>unlocks_class = target_class</code>     | Buying any one class would unlock all of them.                                                    |
| <code>expires_at is null or &gt; now()</code> | Nothing today — every expiry is null. It self-activates the day expiry dates start being written. |

That last one deserves a note. It is written to be **null-tolerant**: null means "no expiry set", so the condition passes and today's behaviour is unchanged. When a payment gateway starts stamping `expires_at`, expiry begins enforcing itself with no code change. Your database README lists "expiry source of truth" as an open decision — this line settles it: **the purchase governs**, and `products.access_until` is catalog metadata to be copied into the purchase at sale time.

## Where target\_class gets its value

The function definition alone gives `target_class` nothing. It is filled at the moment of the call, exactly like a JavaScript argument:

```
function hasClassAccess(targetClass) { ... } // slot is empty
hasClassAccess("class9");                  // now it holds "class9"
```

The caller is the flashcards policy, at the line `has_class_access(chapters.class)`. Following one real card through it:

1. Card's `chapter_slug` is `particulate-nature-of-matter`.
2. Policy looks that up in `chapters`, finds the row, reads its `class` column: `class8`.
3. Policy calls `has_class_access('class8')`.
4. Inside the function, `target_class` is now `'class8'`.
5. It checks for a completed, unexpired purchase of a product with `unlocks_class = 'class8'`.
6. True → the card is returned. False → the card does not exist for this reader.

### THE PRINCIPLE UNDERNEATH ALL OF IT

The class comes out of your own `chapters` table, never out of the request. The URL, the React state, the query string — none of them reach that slot. There is nothing for an attacker to lie about. This single property is what makes it a lock rather than a request.

## Calling it yourself

```
const { data } = await supabase.rpc("has_class_access", { target_class: "class9" });
```

Useful for debugging, and eventually for removing the duplicate purchase logic from `App.jsx` so the UI and the database share one brain. Note that running `select has_class_access('class9');` in the SQL Editor usually returns false regardless — the editor runs as an admin role where `auth.uid()` is null. Test from the app, logged in.

# What happens during one request

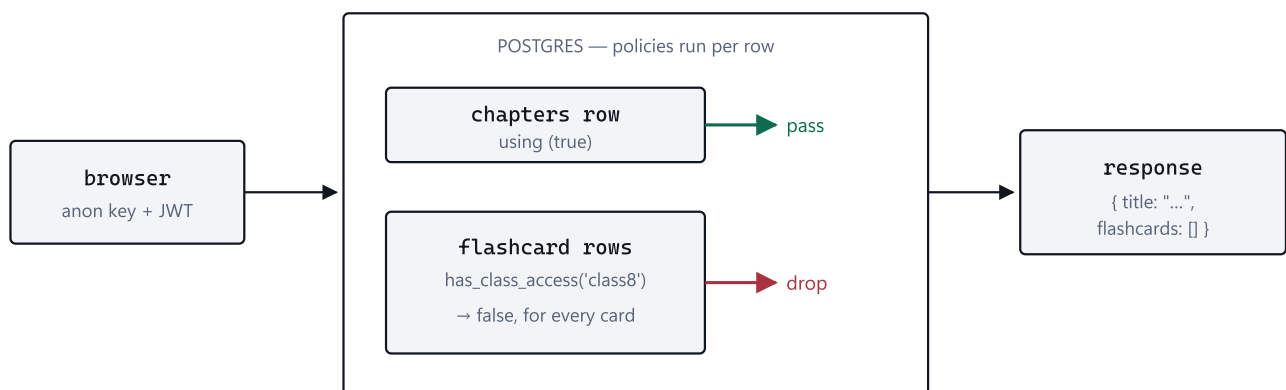
Knowing the pieces is not the same as knowing the sequence. Here is a real request, start to finish.

```
supabase
  .from("chapters")
  .select("title, flashcards(*)")
  .eq("slug", chapterSlug)
  .order("position", { foreignTable: "flashcards" })
  .single();
```

1. The browser sends an HTTP request to your Supabase URL, carrying the public anon key and — if logged in — a signed JWT identifying the user.
2. **PostgREST**, Supabase's API layer, turns that into SQL. It sets the role to `authenticated` and makes the JWT's subject available as `auth.uid()`.
3. Postgres runs the query, and applies RLS policies **per table, per row**.
4. Surviving rows come back as JSON.

## The part that surprises people

This query touches *two* tables, so *two* policies run independently. The chapters policy is `using (true)`, so the chapter row always survives. The flashcards policy runs separately on the embedded cards.



One query, two policies, a partial result. No error is raised — the blocked rows are simply absent.

So a blocked user does **not** get an error. They get:

```
{ title: "Particulate Nature of Matter", flashcards: [] }
```

`.single()` succeeds, because exactly one chapter row came back. `chapter.title` works fine. The problem lands one component deeper: `Flashcard.jsx` sets `deck = []`, so `card` is `undefined`, and rendering `card.question` throws.

## Two different failures, two different guards

| SITUATION                                | WHAT COMES BACK                                                | WHERE IT BREAKS                                       |
|------------------------------------------|----------------------------------------------------------------|-------------------------------------------------------|
| Slug not in <code>chapters</code> at all | <code>.single()</code> errors; <code>chapter</code> stays null | FlashcardlistPage, reading <code>chapter.title</code> |
| Chapter exists, no cards visible         | <code>{ title, flashcards: [] }</code>                         | Flashcard.jsx, reading <code>card.question</code>     |

### THE SECOND ONE IS ALREADY LIVE

Your `src/data` files list every chapter for classes 8, 9, 10 and NEET, but the database holds only what you have seeded. Open any chapter whose cards you have not uploaded and it crashes — today, before RLS is tightened. The policy does not cause this bug; it just adds a second route to it.

One more subtlety for the message you write: because `PurchaseRoute` already redirects non-buyers before this page mounts, the empty-deck case you will actually encounter in normal use is "no cards uploaded yet," not "no access." Word it accordingly.

### A note on loading flags

`loading === false` does not mean "data arrived." It means "we finished trying." Those are different, and conflating them is the most common bug in async React. Any time you set a loading flag to false inside an error branch, check what the render is about to read.

# The React guards, and their limits

Two components guard your routes. Both are useful. Neither is security.

| COMPONENT      | CHECKS                                                                      | OTHERWISE                       |
|----------------|-----------------------------------------------------------------------------|---------------------------------|
| ProtectedRoute | Is there a session?                                                         | Redirect to <code>/login</code> |
| PurchaseRoute  | Does <code>unlockedClasses</code> include the URL's <code>:classId</code> ? | Redirect to <code>/</code>      |

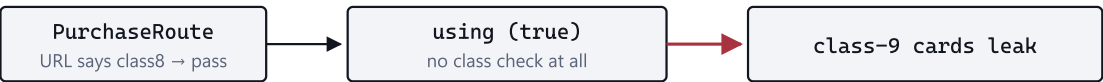
`unlockedClasses` is computed once in `App.jsx`, by querying purchases joined to products and mapping to `unlocks_class`. Both guards render a spinner while `authLoading` is true or `unlockedClasses` is still null — correct, since treating "not yet known" as "not allowed" would flash a redirect on every refresh.

## The gap the database closes

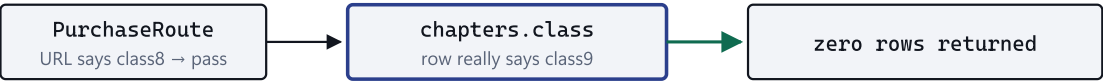
`PurchaseRoute` reads `classId` **from the URL**. But `FlashcardListPage` queries by `chapterSlug` only — it never uses `classId`. Those two facts together make a hole.

A class-8 buyer requests `/home/class8/flashcard/chemistry/<class-9-slug>`

OLD POLICY



NEW POLICY



The guard trusts the URL; the database trusts the row. Only the second survives a tampered address bar.

A class-8 customer can paste a class-9 chapter slug into a URL whose `:classId` still reads `class8`. `PurchaseRoute` checks the URL, sees `class8`, and lets them through. The query then fetches by slug and finds the class-9 chapter. Under the old policy, those cards are returned.

The new policy reads the chapter's *actual* class from the row and asks about that. The guard's opinion is irrelevant. This is the clearest demonstration of why the argument comes from `chapters.class` rather than from `classId`.

## Duplicate logic to keep in mind

`App.jsx` computes access in JavaScript; the function computes it in SQL. Today they agree. They will diverge the moment expiry starts being enforced, because the JavaScript version does not check `expires_at` — the UI would offer a dashboard the database refuses to fill. The eventual fix is to have the client call `has_class_access` over RPC so there is one implementation, not two.

## Two robustness bugs, unrelated to security

- `ChapterlistPage` does `chapterMap[classId][subject]` with no fallback. A URL naming a subject the class does not have — class 8 has no zoology — throws on the second lookup.
- The empty-deck crash from chapter 07 needs a guard, either in `FlashcardlistPage` before rendering, or inside `Flashcard` itself. One place is enough; before rendering is simpler.

# Adding the next content table

PYQ, notes, mock tests — each new table needs its own policy, but **never** new access logic. That distinction is the whole payoff of the design.

## The recipe

1. Create the table with a `chapter_slug` foreign key, exactly like flashcards — including `on delete cascade` and `on update cascade`.
2. `alter table pyqs enable row level security;`
3. Add the adapter policy.
4. Test with an account that owns nothing.

```
create policy "read pyqs for purchased class"
on pyqs for select
to authenticated
using (
  exists (
    select 1 from chapters
    where chapters.slug = pyqs.chapter_slug
    and has_class_access(chapters.class)
  )
);
```

Compare with the flashcards policy: the only difference is `pyqs.chapter_slug` in place of `flashcards.chapter_slug`. The policy is not logic — it is an adapter that says "find this row's class, then ask the function."

A table that stores `class` directly needs no lookup at all:

```
using ( has_class_access(class) );
```

## Why the split pays off

Suppose next year you add a grace period after expiry. You edit the function once, and flashcards, PYQ, notes and everything else enforce it immediately. Had the purchase-checking logic been pasted into each policy, you would be hunting five copies — and the one you missed would not raise an error. It would quietly serve paid content forever.

### THE RULE OF THUMB

Anything that **changes over time** — expiry, refunds, what counts as access — belongs inside the function.

Anything that **multiplies** — one more content table — stays outside as a thin policy. Per-table cost stays constant no matter how many tables you add.

## The one habit that keeps it safe

The function protects nothing unless each new table's policy calls it. A table created in a hurry with `using (true)` leaks everything it holds, silently. Make the ritual automatic: **new content table** → **enable RLS** →



**adapter policy → test with a no-purchase account.**

# Glossary of every keyword used

| TERM                          | MEANING                                                                                                                 |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| primary key                   | The column that uniquely identifies a row. Automatically unique and not null.                                           |
| references                    | Declares a foreign key — this value must exist in the other table. The database refuses data that would break the link. |
| on delete cascade             | Delete the parent row and children go with it.                                                                          |
| on update cascade             | Rename the parent's key and children follow automatically.                                                              |
| not null                      | This column must always have a value.                                                                                   |
| default                       | Value used when an insert does not supply one.                                                                          |
| check ( ... )                 | A rule every row must satisfy. Rejected at write time, so bad data never lands.                                         |
| generated always as identity  | Postgres assigns increasing numbers automatically. The modern replacement for <code>serial</code> .                     |
| numeric(10,2)                 | Exact decimal, 10 digits with 2 after the point. Correct type for money.                                                |
| timestampz                    | A moment in time stored in UTC, converted on read. Always prefer it over <code>timestamp</code> .                       |
| uuid                          | A 128-bit identifier, unguessable — what Supabase Auth uses for user ids.                                               |
| text ~ 'regex'                | Regex match operator.                                                                                                   |
| unique index ... where        | A partial index — uniqueness enforced only on rows matching the condition.                                              |
| index                         | A lookup structure that avoids scanning every row.                                                                      |
| enable row level security     | Switches the table to deny-by-default.                                                                                  |
| create policy                 | Grants back some access under a stated condition.                                                                       |
| for select                    | Which action the policy covers. Others must be granted separately.                                                      |
| to authenticated              | Which database role it applies to. <code>anon</code> is logged-out, <code>authenticated</code> is logged-in.            |
| using ( ... )                 | The true/false test run per row when reading.                                                                           |
| auth.uid()                    | The current user's id, read from their signed token. Null when logged out.                                              |
| exists ( ... )                | True if the subquery returns at least one row. Stops at the first match.                                                |
| select 1                      | Convention inside <code>exists</code> — the value is never used.                                                        |
| join ... on                   | Combines rows from two tables where the condition matches.                                                              |
| language sql / plpgsql        | A single expression, versus a procedural body with <code>begin ... end</code> . Triggers need plpgsql.                  |
| stable / volatile / immutable | How predictable a function is, which decides how often Postgres calls it.                                               |
| security definer              | Run as the function's owner, not the caller. A deliberate privilege escalation.                                         |

| TERM                                          | MEANING                                                                        |
|-----------------------------------------------|--------------------------------------------------------------------------------|
| <code>set search_path</code>                  | Pins schema resolution. Required hygiene with <code>security definer</code> .  |
| <code>\$\$ ... \$\$</code>                    | Dollar quoting — delimits a function body without escaping quotes.             |
| <code>create trigger ... for each row</code>  | Run a function automatically on a table event, once per affected row.          |
| <code>new</code>                              | Inside a row trigger, the row being inserted.                                  |
| <code>-&gt;&gt;</code> and <code>-&gt;</code> | Extract a JSON field as text, versus as JSON.                                  |
| <code>service role key</code>                 | A secret server-side key that bypasses RLS entirely. Never ships to a browser. |

# Open items and known gaps

---

## Do next, in this order

1. **Run the flashcards policy** — as a migration file, per your own convention. Until this runs, every free account can read every card.
2. **Add** `on update cascade` to the flashcards foreign key. Nothing else can be renamed until this exists.
3. **Settle the slug format** and update seeded chapter rows. Cheapest now, at three chapters.
4. **Add the two React guards** — null chapter, and empty deck.
5. **Test with a second account** that owns nothing, including the tampered-URL case from chapter 08.

## Worth doing soon

- Add `set search_path = public` to `handle_new_student`.
- Add `to authenticated` to the students policy, for consistency with the other four.
- Fix the `chapterMap[classId][subject]` crash on an invalid subject in the URL.

## Deferred by design, not oversight

- `role` on `students`, for a future admin dashboard. Additive and non-breaking.
- Payment gateway webhook. When it lands, it writes with the service role key, stamps `purchased_at` and copies `access_until` into `expires_at`. No policy changes required.
- Replacing the JavaScript access calculation in `App.jsx` with an RPC call, so UI and database share one implementation.

## How to verify any of this yourself

Never test access with your own paying account — it proves nothing. The real test is a second account that owns nothing: open DevTools, find the Supabase request, and replay it outside the browser. Zero rows is the only passing result, because that is exactly what an attacker would do.